

Lab o - Sockets

Socket creation and management

GitHub link;

https://github.com/Raablu-afk/Security_Application

GitHub link;	1
Part 1 — Your First Python Program	3
Part 2 — Understanding Sockets	3
Part 4 — Create a Basic TCP Server	4
Part 5 — Create a Basic TCP Client	4
Part 6 — Modify the Client and Server	5
Part 7 — Connect to a Public Web Server	6
Part 8 - Challenge — Rebuild the Client-Server Application Using Socket.IO	7
Part 8 — Security Reflection	8
Reflection	9

Part 1 — Your First Python Program

```
● raph@racbook security application % python3 lab0_test.py  
  My first Python security program  
○ raph@racbook security application %
```

Part 2 — Understanding Sockets

Reflection Question 1

Explain in your own words:

1. What is a socket?

A socket is the interface by which our programs will be able to communicate. By defining a socket for a host or a server, we define how the exchange of information will happen. As far as I understand, we will define the communication protocol (TCP or UDP) and the ip address for the host or destination of our message

2. What does **AF_INET** mean?

This property of the socket defines that we will be using IPV4 to define the addresses of the host.

3. What does **SOCK_STREAM** mean?

The SOCK_STREAM property defines that the hosts will be communication with the TCP protocol instead of the UDP protocol. This also defines that our program does not tolerate loss of information in transit.

Part 4 — Create a Basic TCP Server

Fig.1 - basic server code

```
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 12345
5
6 server_socket = socket.socket(
7     socket.AF_INET,
8     socket.SOCK_STREAM
9 )
10
11 server_socket.bind((HOST, PORT))
12 server_socket.listen(1)
13
14 print("Server is waiting for a connection...")
15
16 client_socket, client_address = server_socket.accept()
17
18 print("Connected by:", client_address)
19
20 data = client_socket.recv(1024)
21
22 message = data.decode()
23
24 print("Client says:", message)
25
26 reply = "Message received"
27
28 client_socket.send(reply.encode())
29
30 client_socket.close()
31 server_socket.close()
```

The present code starts a localhost server that listens for a connection

Fig.2 - execution result

```
○ raph@racbook security application % python3 server.py
Server is waiting for a connection...
□
```

As no client is attempting to connect, the server remains available indefinitely

Part 5 — Create a Basic TCP Client

Fig.3 - Server-side result

```
● raph@racbook security application % python3 server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 50181)
Client says: Hello Server
○ raph@racbook security application % □
```

The server receives the connection and message from the client. The message is displayed

Fig.4 - Client-side result

```
● raph@racbook security application % python3 client.py
Server replied: Message received
○ raph@racbook security application % □
```

The client notifies the user that the server has answered the request

Part 6 — Modify the Client and Server

Task A

Fig.5 - Content modification

<pre>● raph@racbook security application % python3 server.py Server is waiting for a connection... Connected by: ('127.0.0.1', 50207) Client says: This client is Raphael ○ raph@racbook security application %</pre>	<pre>● raph@racbook security application % python3 client.py Server replied: Message received ○ raph@racbook security application %</pre>
---	---

The client now sends a static name to the server as message. This is the information displayed from the server upon execution

Task B + Challenge

Fig.6 - server modification

```
24 print("Client says:", message)
25
26 reply = "You have been authenticated as " + message
27
28 client_socket.send(reply.encode())
```

Fig.7 - response modification

<pre>● raph@racbook security application % python3 server.py Server is waiting for a connection... Connected by: ('127.0.0.1', 50241) Client says: Raphael ○ raph@racbook security application %</pre>	<pre>● raph@racbook security application % python3 client.py Server replied: You have been authenticated as Raphael ○ raph@racbook security application %</pre>
--	---

The modifications applied to the code of the server now uses the data sent by the client to confirm who was authenticated.

Part 7 — Connect to a Public Web Server

Reflection Question 2

1. Which host did the client connect to?

The client connected to a public http server which hosts a simple page publicly used for testing.

2. Which port was used?

Port 80 (http) was used to reach the public web server. Hence, the web page can be accessed through a web browser

Example Domain

This domain is for use in documentation examples without needing permission. Avoid use in operations.

[Learn more](#)

3. Why is port 80 relevant?

For testing purposes, port 80 allows us to receive the information back with no encryption. This helps us learn about sockets and connections without having to manually code the decryption that port 443 (https) would have.

4. What did the server send back?

The server sent back the html code for the web page found on the public server.

```
raph@racbook security application % python3 http_client.py
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 16:53:32 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Wed, 02 Sep 2026 22:14:26 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 13270
cf-cache-status: HIT
CF-RAY: a38ff30fab2230ff-YVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="ico
n" href="data:,"><meta name="viewport" content="width=device-width, initial-scal
e=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-
ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</s
tyle></head><body><div><h1>Example Domain</h1><p>This domain is for use in docum
entation examples without needing permission. Avoid use in operations.</p><p><a
href="https://iana.org/domains/example">Learn more</a></p></div></body></html>

0
```

Part 8 - Challenge — Rebuild the Client-Server Application Using Socket.IO

Fig.8 - Server code using SocketIO

```
server_socketio.py
security application > SocketIO > server_socketio.py > ...
1 from aiohttp import web
2 import socketio
3
4 socket_io = socketio.AsyncServer()
5 app = web.Application()
6
7 socket_io.attach(app)
8 async def index(request):
9     return web.Response(text="Hello world from socketio", content_type='text/html')
10
11 @socket_io.on('message')
12 def print_message(socket_id, data):
13     print("Socket ID: ", socket_id)
14     print("Data: ", data)
15
16 app.router.add_get('/', index)
17
18 if __name__ == '__main__':
19     web.run_app(app)
```

Fig.9 - Client code using SocketIO

```
server.py client.py client_socketio.py
security application > SocketIO > client_socketio.py > ...
1 import socketio
2 sio = socketio.Client()
3
4 @sio.event
5 def connect():
6     print("connection established")
7
8 @sio.event
9 def disconnect():
10    print("disconnected from server")
11
12 sio.connect("http://localhost:8080")
13
14 sio.emit("message", {"data": "my_data"})
15
16 sio.wait()
```

Fig.10 - Server result

```
raph@racbook SocketIO % python3 client_socketio.py
ges/urllib3/__init__.py:35: NotOpenSSLWarning: urlli
b3 v2 only supports OpenSSL 1.1.1+, currently the 's
sl' module is compiled with 'LibreSSL 2.8.3'. See: h
ttps://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
websocket-client package not installed, only polling
transport is available
connection established
^[]
```

From the server-side we can observe that a connection through the established socket has been established

Fig.11 - Client result

```
raph@racbook SocketIO % python3 server_socketio.py
otOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' mo
dule is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/i
ssues/3020
  warnings.warn(
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
Socket ID: iqw3J_oP_3f6MVoYAAAB
Data: {'data': 'my_data'}
```

From the client-side socket, we can observe the return of the information relating to the socket.

Part 8 — Security Reflection

What could happen if a server accepts connections from anyone?

There are several potential issues with a server that accepts requests and answers from anyone. First, our server doesn't verify the content of messages, which makes code injection a real concern. Therefore, after authentication, it's crucial to verify messages to ensure the server's safety.

Is the Lab 0 message encrypted?

The client encodes the message, and the server decodes it, but it's not encrypted or decrypted. This means the message isn't secure during transit.

Could network traffic be observed by another system?

Absolutely. A malicious actor with access to our network or capable of intercepting SIGINT messages while we're on a wireless network could observe the traffic and read the message in plaintext after decoding it.

Why should servers validate input?

This is essential to prevent code injection and cross-site scripting. Currently, our server lacks validation and content, which allows threat actors to bypass its intended purpose. This could enable them to collect system information or pivot to another process.

Why might authentication be useful?

Initial authentication to the server protects any other field from malicious use and safeguards data. If our server were hacked, authentication would prevent non-reputation damage. A user would still need to register their session to access compromised data.

Suggest two ways to make this application more secure.

As previously explained, encryption is the most pressing issue. It's essential before implementing authentication because we can't send passwords and usernames in plaintext. Input validation should also be considered during authentication implementation.

Reflection

A server accepting requests and responses from anyone poses security risks. Message content verification is crucial to prevent code injection attacks. Currently, messages are encoded and decoded without encryption, making them vulnerable to interception and reading, especially on wireless networks. This vulnerability can lead to code injection and cross-site scripting attacks. The server lacks validation and content checks, allowing threat actors to bypass its purpose and collect system information or move to other processes. While initial authentication protects other fields from malicious use and safeguards data, it's not foolproof. If hacked, authentication can prevent non-reputation damage, but users must register their session to access compromised data. Encryption is the most urgent issue, as sending passwords and usernames in plaintext is highly insecure. Implementing encryption is essential before enhancing authentication measures. Input validation should also be considered during authentication to further strengthen security.

.